

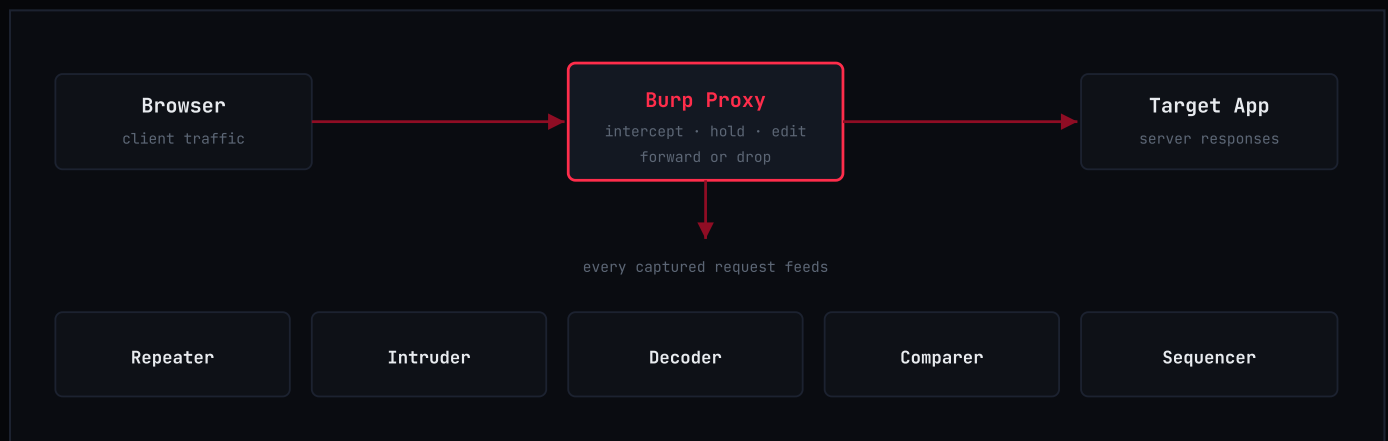
Burp Suite – Field Guide & Quick Reference

Burp Suite (PortSwigger) is the intercepting-proxy platform almost every web application engagement runs through. The core idea is simple: it sits between your browser and the target application, so every request and response passes through it first — which means you can inspect, hold, edit and replay traffic the application never expected a human to touch directly.

Everything else in Burp exists to make that one capability useful. Proxy captures and lets you intercept live traffic. Repeater lets you take a single request and manually poke at it, over and over, until you understand exactly how the server reacts. Intruder automates that poking across large payload sets. Decoder, Comparer and Sequencer are utilities for the fiddly parts — encoding, diffing, and checking whether a token is actually random.

The diagram below is the mental model worth keeping: one proxy, one place traffic flows through, several tools that all draw from the same captured requests. The reference pages that follow are organised the same way — by which tool you're in, not by vulnerability class.

One honesty note: Burp ships as Community (free) and Professional (paid) editions. Everything covered here works in Community. The active vulnerability Scanner and a faster Intruder attack engine are Professional-only and are deliberately left out — this is a reference for the tools you'll actually be clicking, not a pitch for the license.



One proxy in the middle of everything. Every tool downstream works from requests that passed through it.

1. Proxy & Interception

Intercept is on/off — pause traffic before it reaches the target so you can inspect or edit it

Proxy > Intercept

Forward — release the held request or response on to its destination

Forward button

Drop — discard the intercepted item entirely, nothing is sent

Drop button

HTTP history — a running log of every request that has passed through the proxy

Proxy > HTTP history

Intercept Client Requests rules — control which requests actually get held for review

Proxy > Options

Match and Replace — auto-rewrite specific headers or values on every request

Proxy > Options

Intercept Server Responses — hold responses too, not just outbound requests

Proxy > Options

2. Repeater

Send to Repeater — load the current request in for manual, repeated editing

Ctrl+R

Send — fire the request currently loaded in this tab

Ctrl+Space

New tab — keep multiple requests open side by side for comparison

+ button

Render — view an HTML response rendered, not just as raw text

Render tab

Follow redirects — control whether Repeater automatically chases 3xx responses

gear icon

Update Content-Length — keeps a modified body's header honest automatically

on by default

3. Intruder

Send to Intruder — load the current request in for automated payload testing

Ctrl+I

Positions — mark exactly where payloads get inserted with \$ markers

Positions tab

Sniper — one payload set, cycled through each position one at a time

Attack type

Battering ram — the same payload inserted into every position at once

Attack type

Pitchfork — multiple payload sets, marched together position by position

Attack type

Cluster bomb — multiple payload sets, every combination between them tried

Attack type

Grep - Match — flag any response that contains a specific string

Options tab

position →	1	2	3	4	
Sniper					1 set, one position at a time
Battering ram					1 set, same value everywhere
Pitchfork					N sets, marched together
Cluster bomb					N sets, every combination

4. Target, Scope & Site Map

Site map — a tree of everything Burp has passively observed so far Target > Site map

Add to scope — mark a host as in-scope from the site map right-click

Scope settings — define what's actually in play, by URL or host pattern Target > Scope

Show only in-scope items — filter the site map down to what matters filter bar

Engagement tools — Search, Compare site maps, Discover content right-click

Issue definitions — Burp's own reference for what each finding means Target > Issue definitions

5. Decoder

Encode as — URL, HTML, Base64, ASCII hex and more dropdown

Decode as — the reverse of the same transform list dropdown

Smart decode — auto-detect the likely encoding and decode it Smart decode

Hash — compute MD5 / SHA-1 / SHA-256 and others on the current data Hash dropdown

Chain transforms — stack multiple encode/decode passes, each building on the last add row

6. Comparer

Send to Comparer — load two items in from anywhere to diff them right-click

Compare words — diff the two items at the word level Words button

Compare bytes — diff the two items at the byte level Bytes button

Sync scroll — keep both panes aligned while reviewing a long diff on by default

7. Sequencer

Live capture — feed Sequencer live proxy traffic to analyse as it arrives Live capture tab

Manual load — paste in a set of tokens — session IDs, CSRF tokens — to test Paste tab

Analyze now — run the randomness/entropy analysis on the captured set Analyze Now

Character-level analysis — see exactly where predictability creeps into a token results tab

8. Extensions & BApp Store

BApp Store — Burp's built-in extension marketplace [Extensions > BApp Store](#)

Logger++ — richer, filterable request/response logging than the default [BApp Store](#)

Authorize — automated authorization testing across user roles [BApp Store](#)

Turbo Intruder — scriptable, high-throughput alternative to stock Intruder [BApp Store](#)

Param Miner — finds hidden or unlinked parameters and headers [BApp Store](#)

JSON Web Tokens — inspect and tamper with JWTs in place [BApp Store](#)

9. Shortcuts & Workflow

Send to Repeater — the fastest way to start manually probing a request [Ctrl+R](#)

Send to Intruder — the fastest way to start an automated payload run [Ctrl+I](#)

Send (in Repeater) — fire whatever request is currently loaded [Ctrl+Space](#)

Find in view — search within whatever panel currently has focus [Ctrl+F](#)

Right-click > Send to — the same destinations, no shortcut needed [context menu](#)

Scope reminder: *Everything here assumes you're pointed at scope you're actually authorised to test — Burp makes it trivially easy to hammer an endpoint that isn't yours.*